

Applied Big Data Engineering

Mini Project Assessment

Weighting: 25% of Total Module Grade

Type: Group Project (Recommended 2-3 members) or Individual

Duration: 2 Weeks

Project Objective

Students are required to architect and implement an end-to-end Lambda or Kappa architecture data pipeline. The system must simulate a real-world environment where data is ingested in real-time, processed for immediate insights, and orchestrated for historical analysis/reporting.

Preferred Technical Stack

To satisfy the Learning Outcomes, the solution must utilize:

- Ingestion: Apache Kafka (Producers, Topics, Partitions).
- Stream Processing: Apache Spark (Structured Streaming) OR Apache Storm.
- Orchestration: Apache Airflow (for managing batch jobs or reporting pipelines).
- Storage/Sink: A suitable database (PostgreSQL, Cassandra) or File System (HDFS/S3 bucket with Parquet).

Note: You need to justify the tech stack selection against the selected scenario.

Project Overview

In this mini-project you will design and implement a small but complete data platform that ingests data from two kinds of sources: a continuous streaming source and a source that delivers data once a day. Ingested data should be processed to produce a consolidated report or dashboard. Before the implementation, you must decide whether a Lambda architecture or a Kappa architecture is the better fit for your chosen use case, and defend that decision in your report. You need to justify the selected technology stack. What is expected from the final submission is to reason explicitly about architecture, to implement all required ingestion paths, to process the data, and to make the running system observable.

Learning Objectives

- Differentiate Lambda and Kappa architectures and select the appropriate one for a given business problem.
 - Design and justify an end-to-end technology stack across ingestion, processing, storage, and serving layers.
 - Build data pipelines that handle streaming and daily-batch data sources.
 - Apply batch and/or stream processing techniques appropriate to the chosen architecture.
 - Instrument a data pipeline with logging, metrics, and monitoring so that its health and behaviour are observable.
 - Communicate architectural decisions and results clearly in a written technical report.
-

Project Requirements

1. Data Sources (simulated)

Your system must ingest from simulated sources, built as Python scripts:

- Streaming source that continuously emits events (e.g. every few seconds) representing real-time activity relevant to your use case.
- Batch source that uploads/drops a new file (or calls an API) once per simulated day, representing a periodic feed such as a reference dataset, external report, or end-of-day extract.

You may compress simulated time (e.g. one "day" = 5 minutes) so the pipeline can be demonstrated within a reasonable session. State your simulated clock clearly in the report.

2. Processing

- Processing must be consistent with your chosen architecture
- Transformations should be meaningful for the use case (not just pass-through): cleaning, enrichment, joins between the two sources, aggregation, or windowing.

3. Storage & Output

- Processed results must land in a queryable store suited to your serving needs.
- The final deliverable of the running system is a consolidated (daily/hourly) report or dashboard (e.g. a scheduled report file, or a live dashboard) that answers the business question posed by the use case.

4. Observability

Your pipeline must be **observable**, not just functional. At minimum, include:

- **Structured logging across ingestion, processing, and storage stages**
- **At least one basic alert or health-check rule** (e.g. no data received in N minutes, error rate above threshold).

Use Cases - Choose ONE

Each use case requires **two data sources** and poses a business question your consolidated report/dashboard must answer. You may adapt scope (e.g. add/change fields as required)

Use Case 1 Ride-Hailing Fleet Operations

Scenario	A ride-hailing operator wants live visibility into fleet activity and utilization, while reconciling it daily against per-vehicle running costs submitted by fuel providers.
Streaming source	A Python script emitting GPS/telemetry events per trip (trip_id, driver_id, vehicle_id, lat, lon, speed, status [idle/enroute/on_trip], fare, timestamp) every few seconds.
Daily-batch source	A Python script that drops one CSV/JSON file per simulated day containing vehicle expense records from garages and fuel partners (vehicle_id, fuel_cost, maintenance_cost, distance_covered, service_flag).
Business question	What is fleet utilization and earnings by area/time-of-day right now, and which vehicles are becoming unprofitable once yesterday's fuel/maintenance costs are factored in?
Suggested outputs	API endpoint to return real-time fleet utilization metrics (ex: active vehicles, idle ratio, trips/hour, earnings by zone) Threshold-based alerts when a vehicle is considerably idle for a longer period of time A daily per-vehicle profitability reconciliation report.

Use Case 2 Hospital Patient Vital Signs Monitoring

Scenario	A hospital ward wants continuous, near-real-time monitoring of patient vitals from bedside sensors, correlated daily with lab test results that the pathology lab uploads once a day.
Streaming source	A Python script simulates bedside monitors emitting vital-sign readings (patient_id, heart_rate, spo2, systolic_bp, diastolic_bp, temperature, timestamp) every few seconds, including occasional simulated abnormal spikes.
Daily-batch source	A Python script produces one daily file of lab results per patient per simulated day (patient_id, test_type, result_value, reference_range, collected_at).
Business question	Which patients show concerning vital-sign trends right now, and how do yesterday's lab results change the risk picture for those patients going forward?

Suggested outputs	API endpoint to return real-time ward monitoring figures. Threshold-based alerts per patient A daily consolidated patient risk report joining vitals trends with the latest lab results.
--------------------------	--

Use Case 3 Smart Grid Energy Monitoring & Billing

Scenario	A utility company wants real-time visibility into grid load and renewable (solar) contribution from smart meters, reconciled daily against tariff and billing data produced once a day.
Streaming source	A Python script simulating smart meters emitting readings (meter_id, household_id, power_consumption_kwh, solar_generation_kwh, grid_zone, timestamp) at short intervals.
Daily-batch source	A Python script producing one daily file of tariff/billing reference data (household_id, tariff_rate, billing_tier, subsidy_flag) and/or a daily weather-forecast file affecting solar output.
Business question	What is the current grid load and renewable contribution by zone, and what will each household's bill look like once daily tariff data is applied to their consumption?
Suggested outputs	API endpoint to return real-time grid load/renewable-mix by zone Threshold-based alerts when renewable contribution is considerably low. A daily consolidated billing and solar-contribution report per household.

Deliverables

1. Codebase

- Source code for simulated data sources.
- Source code for ingestion, processing, and storage/serving layers.
- Observability configuration (API for metrics export, alert rules).
- A README with architecture summary, setup/run instructions, and how to reproduce results (Docker Compose strongly recommended).
- Inline documentation/comments and, where relevant, automated tests.

2. Report (recommended 8-15 pages)

- Use case chosen and business requirements interpreted from it.
- Architecture decision: Lambda vs Kappa, with explicit justification and rejected alternative.
- Designed architecture diagram(s) covering ingestion, processing, storage, and serving layers.
- Technology stack with justification for each major component.
- Observability design: what is measured, how, and why.
- Results: sample output of the consolidated report/dashboard, with screenshots.
- Limitations, trade-offs, and what you would do differently at production scale.

Submission Guidelines

- Submit a **link to a Git repository** (or a zip archive) containing the full codebase.
- Submit the report as a **PDF**.
- **Include a short (5–10 minute) demo video OR be prepared to demo live, showing the pipeline running end-to-end and the observability results.**
- Clearly state any assumptions, simplifications, or simulated-time compression used.

Marking Rubric - Total 100 Marks

Criterion	Marks	What is assessed
Architecture Decision & Justification (Lambda vs Kappa)	20	Correctness and depth of the argument for the chosen architecture given the use case's latency, replay, cost and consistency requirements; honest discussion of trade-offs and rejected alternatives.
Technology Stack Selection & Justification	10	Appropriateness of chosen tools for each layer; justification tied to use-case constraints rather than generic popularity.
Data Ingestion Implementation	15	Correctness and robustness of simulated sources.
Processing Layer Implementation	15	Correctness of transformation logic; appropriate use of streaming and/or batch processing consistent with the declared architecture
Storage & Serving Layer	10	Correctness of implementation of the serving layer.
Observability	10	Logging, metrics and tracing across pipeline stages to detect and diagnose pipeline failures.
Report	15	Clarity of architecture diagrams, explanation of design decisions, tech stack rationale, and presentation of results; honesty about limitations.
Code Quality & Documentation	5	Readability, modularity, README/setup instructions, configuration management, and reproducibility (e.g. via Docker/Compose).
Total	100	

Notes

- Use of AI coding assistants for boilerplate is permitted; you must be able to explain and defend every architectural decision and every line of core pipeline logic in a viva/demo.
- Group submissions (if assigned) must include a short statement of individual contributions.